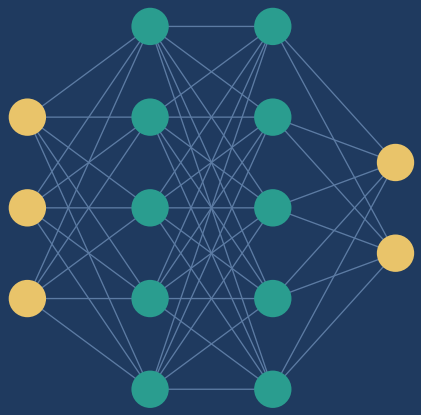
МОДУЛЬ 2 · РАЗГОВОР С МОДЕЛЬЮ · УРОК 5

Техники промптинга

Примеры, рассуждение, цепочки, теги и промпт-ревью



О чём этот урок

В уроке 4 мы разобрали, **из чего** состоит хороший промпт: шесть блоков и шаблон на каждый день. Теперь — **приёмы**, которые выручают, когда шаблона мало: модель путает формат, ошибается в логике, теряется в большой задаче или выдаёт «почти то». Для каждой такой трудности есть своя техника, и все они пишутся обычным языком.

Главное

После урока вы сможете:

- задавать формат и стиль примерами (few-shot) и не попадать в их ловушки;
- просить модель рассуждать по шагам — и понимать, когда это уже не нужно;
- разбивать большую задачу на цепочку промптов с контрольными точками;
- размечать промпт заголовками и тегами и проводить «промпт-ревью» до и после запуска.

Время: ~45 минут чтения + 30 минут практики.

1. От анатомии к технике

Шесть блоков отвечают на вопрос «что сказать модели». Техники отвечают на другой вопрос: **как сказать, чтобы она справилась с конкретной трудностью**. Удобно думать о них как об инструментах в ящике — каждый под свою задачу:

- **Примеры (few-shot)** — когда важны формат, стиль, единообразие.
- **Рассуждение по шагам** — когда есть логика, расчёт, несколько условий.
- **Цепочка промптов** — когда задача большая и состоит из этапов.
- **Структура: заголовки и теги** — когда в промпте много данных и частей.
- **Итерация и промпт-ревью** — когда результат «почти тот» или ошибка стоит дорого.

Все пять техник опираются на то, что мы уже знаем: модель продолжает текст, учитывает всё, что лежит в контекстном окне, и не переспрашивает. Техника просто кладёт в контекст то, что поможет ей не ошибиться.

Частое сомнение

«Промптинг — это набор волшебных фраз: пригрози модели или пообещай чаевые, и она постарается»

Такие советы ходят по интернету, но в аккуратных проверках они не подтверждаются. Работают не заклинания, а то, что даёт модели информацию: образец результата, ход решения, ясные границы данных. Именно этому посвящён урок.

- Угрозы и обещания «чаевых» в целом не дали значимого эффекта на сложных тестах (GPQA[1], MMLU-Pro[2]). На отдельных вопросах результат менялся в обе стороны, и заранее предсказать направление было нельзя. — *Wharton GAIL, Prompting Science Report 3, 2025*

Что здесь справедливо. формулировка действительно влияет на ответ, иногда заметно. Поэтому любую технику, включая описанные здесь, стоит проверить на своих задачах, а не принимать на веру.

В разговоре с коллегой: *«Модели нужны не заклинания, а вводные — те же, что нужны новому сотруднику.»*

[1] **GPQA** — Graduate-Level Google-Proof Q&A: набор очень трудных вопросов по биологии, физике и химии.
[2] **MMLU-Pro** — Massive Multitask Language Understanding, усложнённая версия: тысячи вопросов по десяткам предметов.

2. Примеры: показать вместо объяснять (few-shot)

Вместо того чтобы описывать нужный результат словами, его можно **показать**. Если примеров нет, это zero-shot («с нуля»), один пример — one-shot, несколько — **few-shot**. Техника особенно сильна там, где правило трудно сформулировать, но легко узнать «хорошо» на глаз: классификация, единый формат, фирменный стиль, разбор документов по шаблону.

Пример из практики — разнести проводки по статьям управленческого P&L[3]. Описать все правила долго, а три размеченные строки сразу показывают и логику, и формат ответа.



Рис. 1. Примеры задают и логику, и формат: модель отвечает так же, как в образцах

```
Разнеси проводки по статьям управленческого P&L.
Отвечай только названием статьи из списка: Ремонт вагонов,
Аренда, Страхование, Топливо, Прочее.

Примеры:
Ремонт колёсных пар, вагон 52-114 → Ремонт вагонов
Аренда склада, элеватор «Южный» → Аренда
Страхование груза, рейс 0715 → Страхование

Проводки для разнесения:
Замена тормозных колодок, вагон 61-207
Дизельное топливо, тепловоз ТЭМ-18
```

Как сделать примеры полезными:

- **2–5 разных примеров.** С одним образцом модель копирует его слишком буквально — те же слова и длину. Разные примеры помогают уловить принцип.
- **Покрывайте трудные случаи.** Если есть пограничная проводка (ремонт арендованного вагона — это «Ремонт» или «Аренда»?), покажите, как её решать.
- **Не перекашивайте.** Если из пяти примеров четыре — «Прочее», модель будет чаще отвечать «Прочее». Набор примеров должен отражать реальный разброс.

[3] **P&L** — Profit and Loss: отчёт о прибылях и убытках.

- **Пример не должен спорить с инструкцией.** Если в задаче «до 50 слов», а образец на 200 — модель, скорее всего, поверит образцу.
- **Отделяйте примеры заголовком** («Примеры:») и пустой строкой, чтобы модель не приняла их за данные для обработки. Стрелка «→» — просто разделитель «вход → ответ», подойдёт и двоеточие; главное — одинаково во всех примерах.

Осторожно

Примеры из реальных документов — это реальные данные. Перед тем как вставлять проводки, договоры или отчёты в промпт, обезличьте их (урок 3): названия контрагентов, суммы, номера.

3. Рассуждение по шагам

Вспомним урок 2: модель пишет ответ токен за токеном, и каждый следующий токен опирается на уже написанные. Если попросить сразу ответ, у неё нет «черновика» — вывод появляется до расчёта. Если попросить **сначала рассуждение, потом ответ**, промежуточные шаги попадают в контекст и служат опорой для вывода. Эту технику называют **chain of thought** — цепочка рассуждений.

Пример — проверка ковенанта по долговой нагрузке заёмщика: «Долг / EBITDA[4] ≤ 3,5». Спросим модель двумя способами.

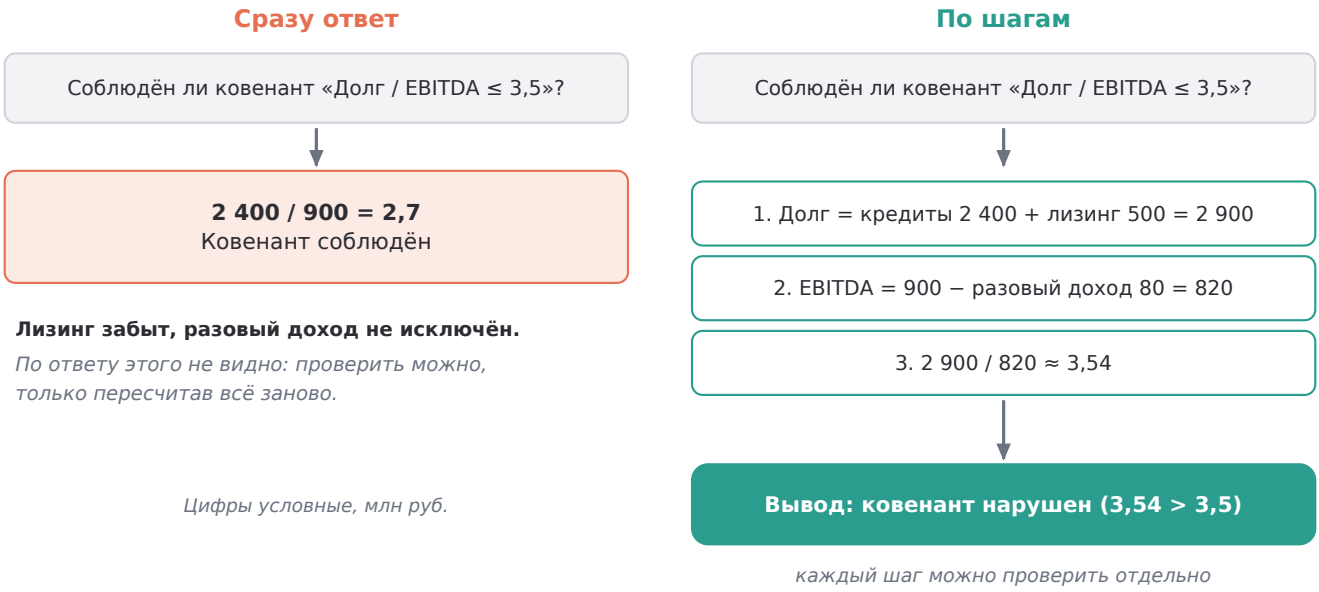


Рис. 2. Один вопрос — два ответа. По шагам видно, что учтено, а что нет

Два способа попросить:

1. **Просто:** «Прежде чем отвечать, рассуди по шагам». Подходит для быстрых вопросов.
2. **Со структурой:** перечислите шаги сами и разделите рассуждение и ответ заголовками — тогда ответ легко найти и проверить.

[4] **EBITDA** — Earnings Before Interest, Taxes, Depreciation and Amortization: прибыль до вычета процентов, налогов и амортизации.

```
Отчётность:
[баланс и отчёт о финансовых результатах]

Проверь, соблюдён ли ковенант «Долг / EBITDA ≤ 3,5».
Сначала рассуждай по шагам под заголовком «Рассуждение»:
1) посчитай долг, включая лизинг;
2) посчитай EBITDA без разовых доходов и расходов;
3) раздели и сравни с порогом.
Затем под заголовком «Ответ»: «соблюдён / нарушен»,
значение и запас.
```

Главная польза для нас даже не в точности, а в **проверяемости**: если ошибка есть, видно, на каком шаге. Неверно посчитан долг — поправили одну строку, а не спорим с итоговой цифрой.

Осторожно

Рассуждение не делает арифметику надёжной. Для точных расчётов лучше попросить модель написать и выполнить код или пересчитать ключевые цифры в Excel (урок 3).

Частое сомнение

«Раз модель расписала рассуждение, значит, ей можно верить»

Рассуждение — это полезный инструмент, но не гарантия. Оно повышает точность и показывает ход решения, однако текст рассуждения — тоже сгенерированный текст. Он не всегда честно отражает, что на самом деле повлияло на ответ.

- Одна фраза «давай рассуждать по шагам» подняла точность модели на школьных задачах с 17,7% до 78,7% (MultiArith[5]) и с 10,4% до 40,7% (GSM8K[6]). *— Kojima и др., NeurIPS 2022*
- Когда рассуждающим моделям подсовывали подсказку к ответу, они признавали это в своём рассуждении лишь в 25% и 39% случаев. *— Anthropic, «Reasoning models don't always say what they think», 2025*

Что здесь справедливо. рассуждение действительно помогает и даёт материал для проверки. Но проверять нужно шаги по исходным данным, а не верить им потому, что они выглядят логично.

В разговоре с коллегой: *«Рассуждение модели — это черновик расчёта, который удобно проверить, а не доказательство.»*

Быстро устаревает — проверяйте актуальное

Многие современные модели «думают» перед ответом сами — в приложениях это режимы рассуждения (их называют thinking, reasoning, «глубокое размышление»). Для таких моделей просьба «рассуждай по шагам» почти ничего не добавляет, а иногда только удлиняет ответ. Перечень шагов, формат и проверка остаются полезными. Какие модели и режимы доступны, меняется каждые несколько месяцев — смотрите документацию сервиса.

[5] **MultiArith** — набор простых текстовых задач по арифметике.
[6] **GSM8K** — Grade School Math 8K: около 8,5 тыс. школьных текстовых задач по математике.

4. Цепочка промптов: большая задача по частям

Когда в одном промпте просят «изучи отчётность, посчитай коэффициенты, сравни с отраслью и напиши заключение», модель старается сделать всё сразу — и где-то срезает углы. Надёжнее разбить работу на **цепочку**: каждый промпт делает одно дело, а его результат становится входом для следующего.



Ошибку видно на том звене, где она возникла, — и исправлять нужно только его

Рис. 3. Кредитное заключение как цепочка из четырёх звеньев

Чем цепочка лучше одного большого промпта:

- **Внимание модели не распыляется:** в каждом звене одна понятная задача.
- **Есть контрольные точки:** вы смотрите таблицу показателей до того, как на ней построят выводы. Ошибка на первом шаге не «протечёт» в заключение.
- **Легко чинить:** не получилось одно звено — переделываете только его.
- **Звенья можно переиспользовать:** промпт «извлеки показатели в таблицу» пригодится для любого заёмщика.

Передать результат между звеньями просто: вставьте его в следующий промпт под понятным заголовком. Например, второе звено получает таблицу из первого:

```
Таблица показателей (результат прошлого шага):
[результат звена 1]

Исходная отчётность:
[исходный документ]

Сверь каждое число в таблице с отчётностью. Выведи только
расхождения: показатель | в таблице | в отчётности | страница.
Если расхождений нет — напиши «расхождений нет».
```

Аналогия

Цепочка похожа на то, как готовят материалы к кредитному комитету: один сотрудник собирает данные, второй их сверяет, третий считает, четвёртый пишет записку. Никто не делает всё сразу, и на каждом этапе работу можно принять или вернуть.

Когда цепочка не нужна: если задача умещается в одно действие (переписать письмо, перевести абзац) или если ошибку легко увидеть в итоговом ответе. Правило то же, что в уроке 4: **чем дороже ошибка, тем больше структуры**.

5. Структура: от заголовков к тегам

Во всех примерах выше части промпта отделялись **заголовками**: «Примеры:», «Проводки для разнесения:», «Рассуждение». Это и есть структура — промпт как документ с разделами. Для большинства задач её хватает: короткий заголовок, двоеточие, пустая строка между частями.

Когда заголовков мало

Граница между частями размывается, когда частей много, они вложены друг в друга (два года отчётности, пять вагонов) или внутри самих данных встречаются похожие слова — например, в отчёте есть свой раздел «Примечания:». Тогда помогают **теги**: метка в начале блока и метка в конце.

Заголовком	Тегами
Отчётность за 2024: [данные]	<отчётность_2024> [данные] </отчётность_2024>
Где кончается блок, модель угадывает по пустой строке и смыслу	Где кончается блок, видно точно — по закрывающей метке
Хватает почти всегда	Нужны, когда частей много, есть вложенность или ответ идёт дальше

Главное

Теги — это не программирование. Это пара меток: <название> в начале блока и </название> в конце; косая черта означает «блок закончился». Названия вы придумываете сами, по-русски или по-английски, — специального синтаксиса нет. Одну такую метку, <отчётность>, вы уже видели в уроке 4.

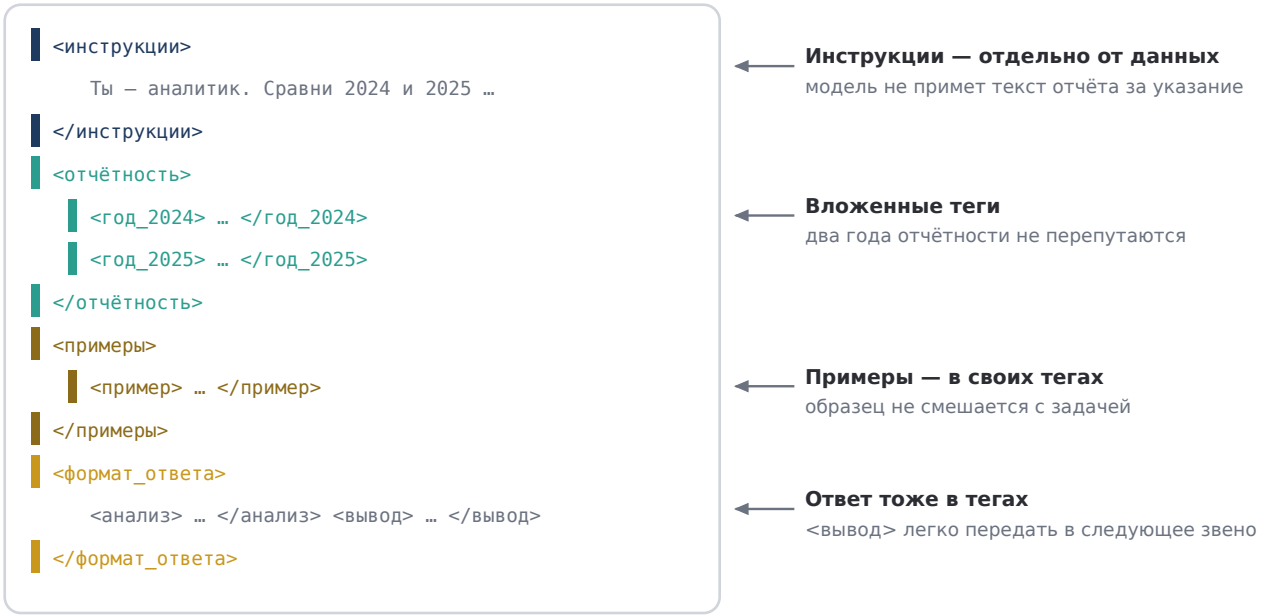


Рис. 4. Большой промпт, размеченный тегами: у каждой части своя «папка»

Правила простые:

- **Понятные названия:** <отчётность>, <договор>, <пример>, а не <a1>. В тексте задачи ссылайтесь на них: «по данным из <отчётность>».
- **Вложенность для однотипных частей:** два года, три контрагента, пять вагонов — каждый в своём теге внутри общего.
- **Попросите ответ в тегах:** «рассуждение — в <анализ>, итог — в <вывод>». Так ответ проще прочитать глазами, вставить в следующее звено цепочки или разобрать скриптом.
- **Без фанатизма:** для короткого промпта хватит заголовков, а для вопроса без данных — и их не нужно.

```
<вагоны>
<вагон> номер 52-114, пробег, даты ремонтов, простой … </вагон>
<вагон> номер 61-207, … </вагон>
</вагоны>

По данным из <вагоны> найди вагоны, у которых простой в ремонте
за квартал больше 10 дней. Для каждого — номер, дни простоя и
вероятная причина, если она видна из данных.
Ответ — в <таблица>, комментарий на 2–3 строки — в <вывод>.
```

6. Итерации и промпт-ревью

Из урока 4 мы знаем: работа с ИИ — цикл, а не выстрел. Здесь добавим к циклу два приёма **ревью** — проверки силами самой модели, до запуска и после.



Обе петли — по 1–2 круга. Факты и расчёты сверяйте с данными, а не с мнением модели

Рис. 5. Две петли проверки: промпт до запуска и ответ после

6.1. Итерация с конкретной обратной связью

«Переделай» почти ничего не говорит модели. Говорите, **что именно** не так и **как** должно быть: «Третий риск — общие слова. Замени его риском, который виден из отчётности, со ссылкой на показатель». Если после 2–3 правок разговор ушёл в сторону, поправьте исходный промпт и начните новый чат.

6.2. Ревью промпта — до запуска

Попросите модель проверить ваш промпт глазами нового сотрудника. Делайте это в отдельном чате, чтобы ревьюер не начал сразу выполнять задачу:

```
Ниже промпт, который я собираюсь использовать. Не выполняй его.
Прочитай как новый сотрудник, который впервые видит задачу:
1) что в нём неоднозначно или можно понять двояко;
2) каких сведений не хватает;
3) предложи исправленную версию.

<промпт>
[ваш промпт]
</промпт>
```

6.3. Ревью ответа — после запуска

Проверка ответа сильна, когда у неё есть **критерии и опора**. Не «проверь себя», а «проверь по чек-листу и по исходным данным»:

```
<отчётность> … </отчётность>
<заключение> … </заключение>

Проверь заключение по пунктам:
1) каждое число есть в отчётности (укажи, где);
2) каждый вывод следует из чисел;
3) названы риски по долговой нагрузке и ликвидности.
Выведи только найденные проблемы. Если их нет — так и напиши.
```

Частое сомнение

«Проверять ИИ тем же ИИ бессмысленно: он будет хвалить сам себя»

Отчасти верно — и поэтому важно, **как** устроена проверка. Самокритика по критериям заметно улучшает текст: стиль, полноту, соответствие формату. А вот найти собственную ошибку в логике «из головы» модель часто не может — ей нужна внешняя опора: исходные данные, код, человек.

- Цикл «черновик → самокритика → правка» улучшил результаты в среднем примерно на 20 процентных пунктов на семи задачах, хотя в математике прирост был почти нулевым. — *Madaan и др., Self-Refine, NeurIPS 2023*
- В задачах на рассуждение без внешней обратной связи модели не исправляли свои ошибки, а иногда после «самопроверки» отвечали хуже. — *Huang и др., ICLR 2024*

Что здесь справедливо. самопроверка не заменяет внешнюю. Факты и расчёты сверяйте с документом или кодом, а решение по-прежнему за вами (урок 3, матрица доверия).

В разговоре с коллегой: *«Модель хорошо вычитывает текст по чек-листу, но цифры мы сверяем с отчётностью сами.»*

6.4. Блокнот промптов

Удачный промпт — рабочий актив, как шаблон Excel. Храните его вместе с датой, задачей и пометкой, что меняли и почему. Через месяц у вас будет библиотека проверенных звеньев для цепочек: «извлечь показатели», «сверить», «написать вывод для комитета».

Быстро устаревает — проверяйте актуальное

В ряде сервисов для разработчиков есть встроенные генераторы и «улучшатели» промптов — например, в консоли Anthropic и в OpenAI Playground. Они делают по сути то же ревью промпта. Набор таких функций и их названия часто меняются.

7. Как выбрать технику

Не нужно применять всё сразу. Начинайте с обычного промпта по шаблону из урока 4, посмотрите на ответ и подберите технику под то, что пошло не так:



Техники сочетаются: цепочка, в каждом звене — структура, примеры и проверка

Рис. 6. Шпаргалка: симптом → техника

Задача	Сочетание техник
Разнести 300 проводок по статьям P&L	Примеры + теги для данных + выборочная проверка
Проверить ковенанты заёмщика	Рассуждение по шагам + расчёт кодом + проверка по отчётности
Кредитное заключение	Цепочка из 3–4 звеньев + теги + ревью ответа по чек-листу
Регулярный отчёт по простоям вагонов	Шаблон + теги + пример хорошего отчёта в блокноте

Частое сомнение

«Модели скоро поумнеют, и все эти техники станут не нужны»

Часть приёмов действительно уходит: модели сами рассуждают, лучше понимают намерения и меньше зависят от формулировок. Но модель по-прежнему не знает ваших данных, стандартов и цели — это нужно сообщить. Ясная постановка, образец результата и проверка не устаревают.

- Просьба рассуждать по шагам дала обычным моделям небольшой прирост точности, но добавила разброса в ответах. Рассуждающим моделям она почти не помогла, а время ответа выросло на 20–80%. *— Wharton GAIL, Prompting Science Report 2, 2025*

Что здесь справедливо. конкретные «трюки» стареют быстро, и тратить время на коллекцию волшебных формулировок не стоит. Поэтому в курсе они вынесены в фиолетовые плашки, а в основном тексте — принципы.

В разговоре с коллегой: *«Трюки устаревают, а умение поставить задачу и проверить результат — нет.»*

Глоссарий урока

Термин	Простыми словами
Zero-shot	Промпт без примеров: только описание задачи
One-shot / few-shot	Промпт с одним / несколькими образцами желаемого результата
Chain of thought (цепочка рассуждений)	Просьба сначала рассуждать по шагам, потом отвечать
Рассуждающая модель	Модель, которая сама «думает» перед ответом, без особой просьбы
Цепочка промптов (prompt chaining)	Большая задача, разбитая на звенья: выход одного — вход следующего
Декомпозиция	Разбиение задачи на простые шаги
Контрольная точка	Место в цепочке, где человек проверяет промежуточный результат
Теги (разметка)	Метки вида <данные>…</данные>, делящие промпт на части
Промпт-ревью	Проверка промпта моделью до запуска: что неясно, чего не хватает
Самокритика (self-refine)	Модель оценивает свой ответ по критериям и исправляет его
Блокнот промптов	Личная библиотека проверенных промптов с пометками

Вопросы для самопроверки

1. Чем few-shot отличается от zero-shot и когда примеры полезнее описания?
2. Почему пять примеров, из которых четыре с ответом «Прочее», — плохой набор?
3. Зачем модели рассуждать «вслух», если она и так умеет считать?
4. Модель подробно расписала рассуждение. Достаточно ли этого, чтобы доверять выводу?
5. Назовите два преимущества цепочки промптов перед одним большим промптом.
6. Когда для разметки промпта хватит заголовков, а когда лучше теги?
7. Чем «проверь себя» отличается от хорошего ревью ответа?

Ответы для проверки

Ответы

1. Zero-shot — только описание, few-shot — описание плюс образцы. Примеры полезнее, когда правило трудно сформулировать, но легко показать: формат, стиль, классификация.
2. Модель перенимает перекос и начнёт чаще отвечать «Прочее». Примеры должны отражать реальный разброс и покрывать трудные случаи.
3. Модель пишет ответ по токенам; промежуточные шаги в тексте становятся опорой для вывода. А нам шаги позволяют увидеть, где ошибка.
4. Нет. Рассуждение — тоже сгенерированный текст и не всегда отражает реальные причины ответа. Шаги нужно сверить с данными, а расчёты — пересчитать.
5. Например: в каждом звене одна задача; есть контрольные точки; сломанное звено переделывается отдельно; звенья можно переиспользовать.
6. Заголовков хватает почти всегда. Теги нужны, когда частей много, есть вложенность (несколько лет, вагонов, контрагентов) или ответ передаётся в следующее звено: граница блока видна точно.
7. Хорошее ревью опирается на критерии (чек-лист) и на исходные данные, а не на «мнение» модели о своём же ответе.

Практика

Практика

Задание 1. Классификатор на примерах (10 мин)

Возьмите 10 обезличенных проводок или строк выписки. Напишите промпт со списком статей и тремя разными примерами (как в разделе 2). Затем уберите примеры и сравните: где ответы стали менее единообразными?

Задание 2. Сразу или по шагам (10 мин)

Задайте модели задачу дважды — сначала «дай только ответ», затем с рассуждением по шагам:

```
Вагон за месяц: 6 дней в погрузке, 9 дней в пути с грузом,
7 дней порожнего пробега, 4 дня в ремонте, остальное — простой.
Месяц — 30 дней. Какую долю месяца вагон шёл с грузом
и сколько дней простаивал?
```

Проверьте оба ответа вручную. Где проще найти ошибку, если она есть?

Практика

Задание 3. Цепочка для своей задачи (15 мин)

Возьмите регулярную задачу (отчёт, справка, анализ) и разбейте её на 3 звена. Для каждого запишите: что на входе, что на выходе, где контрольная точка. Прогоните цепочку один раз.

Задание 4. Промпт-ревью (5 мин)

Отправьте свой шаблон из практики урока 4 на ревью по образцу из раздела 6.2. Какие неоднозначности нашла модель? Согласны ли вы с ними? Сохраните улучшенную версию в блокнот промптов.

Что почитать

- Официальное руководство Anthropic по промптингу — docs.claude.com, разделы о примерах, рассуждении, XML[7]-тегах и цепочках промптов.
- Wharton Generative AI Labs, серия «Prompting Science Reports» — короткие проверки популярных советов по промптингу на данных.
- Kojima и др., «Large Language Models are Zero-Shot Reasoners» (2022) — та самая статья про «давай рассуждать по шагам».
- Следующий урок: **«Документы и данные»** — как давать модели отчёты, таблицы и PDF, и как не потерять в них цифры.

Источники фактов

Ссылки кликабельны. Реестр всех фактов урока — в файле docs/facts.md репозитория.

- Meincke L., Mollick E., Mollick L., Shapiro D. Prompting Science Report 3: I'll pay you or I'll kill you — but will you care? // Wharton GAIL, 2025 — gail.wharton.upenn.edu/research-and-insights/techreport-threaten-or-tip · arxiv.org/abs/2508.00614
- Kojima T. и др. Large Language Models are Zero-Shot Reasoners // NeurIPS 2022 — arxiv.org/abs/2205.11916
- Anthropic. Reasoning models don't always say what they think, 03.04.2025 — anthropic.com/research/reasoning-models-dont-say-think
- Madaan A. и др. Self-Refine: Iterative Refinement with Self-Feedback // NeurIPS 2023 — arxiv.org/abs/2303.17651
- Huang J. и др. Large Language Models Cannot Self-Correct Reasoning Yet // ICLR 2024 — arxiv.org/abs/2310.01798
- Meincke L., Mollick E., Mollick L., Shapiro D. Prompting Science Report 2: The Decreasing Value of Chain of Thought in Prompting // Wharton GAIL, 2025 — gail.wharton.upenn.edu/research-and-insights/tech-report-chain-of-thought · arxiv.org/abs/2506.07142

[7] **XML** — Extensible Markup Language: язык разметки данных тегами; теги в промптах похожи на него внешне.